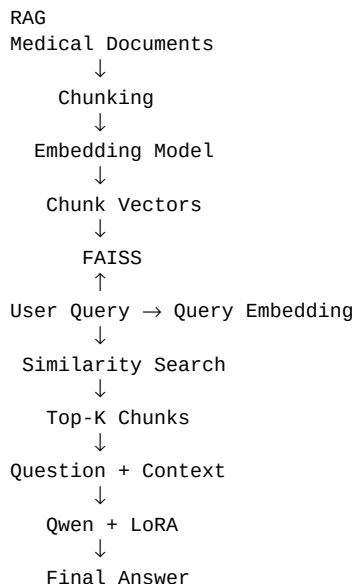


MedAI: Complete Training + RAG Flow

Qwen2.5-7B-Instruct • QLoRA • FAISS • Retrieval-Augmented Generation

Purpose: A step-by-step study guide explaining every stage in your complete project flow, including definitions, what happens internally, why each component is used, and its significance.

1. YOUR COMPLETE FLOW



Core idea: Fine-tuning teaches Qwen the task/style using the training data. RAG retrieves external knowledge at inference time. FAISS retrieves; Qwen generates.

2. TRAINING PHASE — STEP BY STEP

Step 1: Load MedQuAD Dataset

Definition: Load the medical question-answer dataset before training.

What happens: Read fields such as qtype, Question and Answer. The dataset supplies examples for the medical QA task.

Significance: Without quality examples, the model cannot learn the desired task behavior.

Key terms: Dataset = collection of training examples; MedQuAD = medical QA dataset.

Interview line: I first load the MedQuAD dataset and use it as the source for fine-tuning.

Step 2: Data Cleaning

Definition: Prepare the raw dataset by handling problematic records.

What happens: Check null/empty values, duplicates, extra whitespace and invalid text. Do not remove every symbol blindly because medical symbols can be meaningful.

Significance: Cleaner data reduces noise and improves training quality.

Key terms: Null = missing value; duplicate = repeated record; cleaning = preprocessing raw data.

Interview line: I clean missing, duplicate, empty and formatting-problematic records before training.

Step 3: Instruction Formatting

Definition: Convert each QA pair into the conversational format expected for instruction following.

What happens: Question becomes the user message and answer becomes the assistant response, typically using Qwen's chat template.

Significance: Teaches the model the desired question-to-answer behavior.

Key terms: Instruction tuning = training a model to follow task instructions; chat template = standard user/assistant structure.

Interview line: I format the medical QA examples as user instructions and assistant responses.

Step 4: Train / Validation / Test Split

Definition: Divide data into training, validation and test sets.

What happens: Train updates parameters; validation monitors/tunes training; test is reserved for final evaluation.

Significance: Separates learning from evaluation and helps measure generalization.

Key terms: Train = learning; validation = monitoring/tuning; test = final evaluation.

Interview line: Training teaches, validation guides, and testing evaluates.

Step 5: Tokenization

Definition: Convert text into tokens and token IDs.

What happens: Qwen's tokenizer breaks text into token/subword units and maps them to integer IDs.

Significance: Neural networks require numerical representations rather than raw text.

Key terms: Tokenizer = text-to-token converter; token ID = integer representing a vocabulary token.

Interview line: Text is converted into token IDs using Qwen's tokenizer.

Step 6: Embedding Layer

Definition: Map token IDs to dense vectors using Qwen's learned embedding matrix.

What happens: Each token ID indexes a vector that becomes the model's initial representation of that token.

Significance: Transformers process vectors, not raw integer IDs.

Key terms: Embedding = dense vector representation; embedding matrix = learned token-to-vector table.

Interview line: Token IDs are indices; the embedding layer converts them into vectors.

Step 7: RoPE

Definition: Rotary Positional Embedding adds positional information to attention representations.

What happens: Qwen2.5 uses RoPE rather than simply adding a positional vector to the token embedding. RoPE rotates Q/K representations to encode position information.

Significance: Allows the model to understand token order and relative positions.

Key terms: RoPE = Rotary Positional Embedding; Q = Query; K = Key.

Interview line: Qwen2.5 uses RoPE for positional information; it is not simply added as a normal positional embedding.

Step 8: Transformer Blocks

Definition: Repeated neural-network blocks that transform token representations into contextual hidden states.

What happens: Qwen2.5-7B has 28 Transformer blocks and a hidden size of 3584. Blocks contain self-attention, feed-forward processing, normalization and residual connections.

Significance: This is the main language-processing engine of Qwen.

Key terms: Self-attention = tokens interact using Q/K/V; FFN = feed-forward network; hidden state = contextual vector.

Interview line: The Transformer blocks build contextual representations from the token sequence.

Step 9: Causal Language Modeling

Definition: A causal language model predicts the next token from previous/context tokens.

What happens: Qwen predicts the next token while causal masking prevents a position from using future target tokens during training.

Significance: Enables autoregressive text generation.

Key terms: Causal LM = next-token prediction; autoregressive = generate one token at a time.

Interview line: Qwen is a decoder-only causal language model trained for next-token prediction.

Step 10: 4-bit Quantization

Definition: Represent the frozen base-model weights using a low-bit representation, commonly NF4 in QLoRA.

What happens: The pretrained Qwen weights are quantized to reduce their memory footprint. This is primarily a memory-saving technique, not simply a speed trick.

Significance: Makes large-model fine-tuning feasible on smaller GPU memory.

Key terms: Quantization = lower-bit weight representation; NF4 = NormalFloat4; 4-bit = low-bit storage.

Interview line: 4-bit quantization reduces the memory required by the frozen base model.

Step 11: LoRA

Definition: Add small trainable low-rank matrices instead of directly updating the full pretrained matrices.

What happens: For a weight W , the effective weight can be viewed as $W' = W + \Delta W$, with $\Delta W = BA$. W is frozen; A and B are trainable.

Significance: Only a tiny fraction of parameters are trained, saving memory and computation.

Key terms: LoRA = Low-Rank Adaptation; A/B = trainable low-rank matrices; PEFT = Parameter-Efficient Fine-Tuning.

Interview line: LoRA learns a small update to pretrained weights while the original weights remain frozen.

Step 12: QLoRA

Definition: Combine 4-bit quantization of the frozen base model with LoRA fine-tuning.

What happens: Qwen base weights are quantized and frozen; LoRA adapters are trained on top.

Significance: Combines memory-efficient model storage with parameter-efficient fine-tuning.

Key terms: QLoRA = Quantized Low-Rank Adaptation; adapter = small trainable parameters.

Interview line: QLoRA means a quantized frozen base model plus trainable LoRA adapters.

Step 13: Forward Pass

Definition: Run an input through the model to obtain predictions.

What happens: Tokens move through embeddings, RoPE, Transformer blocks and the LM head to produce vocabulary logits.

Significance: Creates the prediction that will be evaluated against the target.

Key terms: Forward pass = input-to-output computation; prediction = model output.

Interview line: The forward pass produces next-token predictions.

Step 14: LM Head and Logits

Definition: Map the final hidden representation to a score for every vocabulary token.

What happens: The LM head projects the hidden state to vocabulary size, producing raw logits.

Significance: Connects the internal representation to candidate output tokens.

Key terms: LM head = output projection; vocabulary = all possible tokens; logits = raw scores.

Interview line: The LM head converts final hidden states into vocabulary-sized logits.

Step 15: Softmax and Decoding

Definition: Convert logits into probabilities and use a decoding strategy to choose the next token.

What happens: Softmax produces probabilities. Greedy decoding selects the highest probability; other strategies include sampling, top-k and top-p.

Significance: Turns numerical scores into generated tokens.

Key terms: Softmax = logits-to-probabilities; decoding = selecting the next token.

Interview line: Softmax gives probabilities; decoding selects the next token.

Step 16: Cross-Entropy Loss

Definition: Measure how well the predicted probability distribution assigns probability to the correct target token.

What happens: The predicted token probabilities are compared with the target token at each position.

Significance: Provides the numerical error signal used for learning.

Key terms: Loss = prediction error; cross-entropy = standard next-token classification loss; target token = correct token.

Interview line: Loss compares predicted token probabilities with target tokens, not with embedding vectors.

Step 17: Backpropagation

Definition: Compute gradients of the loss with respect to trainable parameters.

What happens: The loss signal is propagated backward through the computation graph. In QLoRA, trainable LoRA parameters receive gradients while frozen base weights are not updated.

Significance: Tells the optimizer how trainable parameters should change.

Key terms: Gradient = derivative of loss with respect to a parameter; backpropagation = gradient-computation procedure.

Interview line: Backpropagation computes gradients for the trainable LoRA parameters.

Step 18: 8-bit AdamW

Definition: Use AdamW with memory-efficient 8-bit optimizer states to update trainable parameters.

What happens: AdamW uses the gradients to update LoRA weights; 8-bit optimizer states reduce optimizer memory.

Significance: Helps fit fine-tuning into limited GPU memory.

Key terms: AdamW = adaptive optimizer with decoupled weight decay; optimizer = parameter-update algorithm.

Interview line: 8-bit AdamW updates LoRA parameters while reducing optimizer-state memory.

Step 19: Training Loop and Trained Adapter

Definition: Repeat forward pass, loss, backpropagation and optimizer update over batches and epochs.

What happens: After training, save the learned LoRA adapter. The pretrained Qwen base weights remain frozen.

Significance: Produces a task-specific adapter that can later be loaded with Qwen.

Key terms: Batch = group of examples; epoch = one pass through training data; adapter = saved LoRA parameters.

Interview line: After repeated updates, the trained LoRA adapter contains the task-specific learning.

3. RAG PHASE — STEP BY STEP

Step 20: Load Medical Documents

Definition: Collect external knowledge sources that the chatbot should retrieve from.

What happens: Load approved medical documents, articles, guidelines or other relevant sources.

Significance: Provides external knowledge beyond what is stored in the model parameters.

Key terms: Corpus = collection of documents; document = source text.

Interview line: RAG starts with a knowledge corpus from which relevant information can be retrieved.

Step 21: Chunking

Definition: Split large documents into smaller retrieval units called chunks.

What happens: Divide documents using a chosen chunk size, optionally with overlap between neighboring chunks.

Significance: Smaller chunks improve retrieval precision and keep the context manageable.

Key terms: Chunk = small document piece; chunk size = target/max chunk length; overlap = repeated boundary text.

Interview line: Chunking makes large documents searchable at a useful retrieval granularity.

Step 22: Chunk Embeddings

Definition: Convert each chunk into a semantic vector using an embedding model.

What happens: The embedding model maps each chunk to a vector so semantically similar texts can be close in vector space.

Significance: Enables semantic retrieval rather than relying only on exact keyword matches.

Key terms: Embedding model = text-to-semantic-vector model; semantic vector = numerical meaning representation.

Interview line: RAG embeddings are for retrieval and are separate from Qwen's internal token embedding layer.

Step 23: FAISS Index

Definition: Store chunk vectors in a similarity-search index.

What happens: Add embeddings to FAISS and maintain a mapping from vector IDs to the original chunk text and metadata.

Significance: Allows efficient nearest-neighbor search over many vectors.

Key terms: FAISS = Facebook AI Similarity Search; vector ID = identifier; metadata = source information.

Interview line: FAISS stores/searches vectors; it does not generate the answer.

Step 24: User Query Embedding

Definition: Convert the user's question into the same retrieval vector space as the document chunks.

What happens: Pass the question through the same embedding model used for the chunks.

Significance: Makes the query comparable to stored document vectors.

Key terms: Query embedding = vector representation of the question.

Interview line: The query and documents must use compatible embeddings for meaningful similarity search.

Step 25: Similarity Search

Definition: Find the stored chunk vectors most similar to the query vector.

What happens: FAISS compares the query vector with indexed vectors using the configured similarity/distance metric and returns nearest IDs and scores/distances.

Significance: Identifies the most relevant knowledge pieces.

Key terms: Similarity = numerical closeness; Top-K = K retrieved results.

Interview line: FAISS performs similarity search and returns the most relevant vector IDs.

Step 26: Top-K Chunks

Definition: Recover the actual text corresponding to the top-K vector IDs.

What happens: Use the returned FAISS IDs to retrieve the original chunks and their metadata.

Significance: Converts numerical search results into useful textual context.

Key terms: Top-K = selected K chunks; context = information supplied to Qwen.

Interview line: FAISS returns IDs; the application maps those IDs back to the actual chunk text.

Step 27: Context Augmentation

Definition: Combine the user question with the retrieved chunks in the prompt.

What happens: Construct a prompt containing relevant context plus the user's question and instructions.

Significance: Gives Qwen external evidence to use while generating the response.

Key terms: Augmentation = adding retrieved information; grounding = answering using supplied context.

Interview line: The retrieved context augments the original question before generation.

Step 28: Qwen + LoRA Generation

Definition: Use the fine-tuned Qwen model to generate an answer from the question and retrieved context.

What happens: Qwen processes the augmented prompt and autoregressively predicts output tokens using the trained LoRA adapter.

Significance: Combines task-specific fine-tuning with externally retrieved knowledge.

Key terms: Inference = using the trained model; generation = autoregressive token prediction.

Interview line: FAISS retrieves the information; Qwen uses it to generate the answer.

Step 29: Final Answer

Definition: Decode the generated token IDs into readable text.

What happens: Qwen generates tokens until a stopping condition, and the tokenizer converts the token IDs back to text.

Significance: Produces the final user-facing response.

Key terms: EOS = end-of-sequence; decoding = token IDs to text.

Interview line: The final answer is generated by Qwen, not by FAISS.

4. CRITICAL DISTINCTIONS TO REMEMBER

Token embedding vs RAG embedding

Qwen token embedding: token ID → vector for the language model. RAG embedding: whole chunk/query → semantic vector for retrieval.

FAISS vs Qwen

FAISS retrieves relevant information. Qwen generates natural-language output.

Pretrained weights vs LoRA

Pretrained Qwen weights were learned during pretraining and are frozen during QLoRA. LoRA A/B matrices are trainable.

4-bit vs 8-bit

4-bit quantization mainly reduces base-model weight memory. 8-bit AdamW mainly reduces optimizer-state memory.

Logits vs probabilities

Logits are raw vocabulary scores; softmax converts them into probabilities.

Softmax vs decoding

Softmax creates probabilities; decoding chooses the next token.

Loss vs backpropagation vs optimizer

Loss measures error; backpropagation computes gradients; the optimizer updates trainable parameters.

Training vs RAG

Fine-tuning learns a task-specific adapter. RAG retrieves external knowledge at inference time.

5. END-TO-END EXAMPLE

TRAINING:

Question → Tokenizer → Token IDs → Embeddings → RoPE
→ Transformer → LM Head → Logits → Softmax
→ Target-token comparison → Cross-Entropy Loss
→ Backpropagation → LoRA update → Trained Adapter

RAG:

Documents → Chunks → Embedding Model → Chunk Vectors → FAISS

User Question → Query Embedding → Similarity Search
→ Top-K Chunks → Question + Context
→ Qwen + LoRA → Generated Answer

Simple mental model: Training teaches the model *how to answer the task*. RAG supplies *what information to use right now*. FAISS retrieves; Qwen generates.

6. FINAL INTERVIEW ANSWER

Training: I cleaned and instruction-formatted the MedQuAD dataset, tokenized it with Qwen's tokenizer, and fine-tuned Qwen2.5-7B-Instruct using QLoRA. The frozen base model was quantized to reduce memory, while trainable LoRA adapters were updated using backpropagation and an optimizer such as 8-bit AdamW.

RAG: I split medical documents into chunks, embedded the chunks, and indexed the vectors in FAISS. At inference, the user query is embedded and used for similarity search. The top-K retrieved chunks are added to the prompt, and the fine-tuned Qwen model generates the final answer using that context.